

Audio Deepfake Detection Using Classical Digital Signal Processing

Omar Shoubaki, Omar Rabieh

ID: 1230329, ID: 1230559

DSP Course Project

Section: 1

Abstract

This project presents a classical digital signal processing (DSP) system for detecting AI-generated, or deepfake, speech. The system preprocesses each audio file, applies short-time frequency analysis, uses a digital filter, and extracts DSP features to decide whether the audio is real or fake. We tested four classifiers: Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Logistic Regression, and Decision Tree. The balanced dataset contains real LibriSpeech recordings and fake speech generated using OpenAI TTS, with all files converted to 16 kHz. We first used seven basic DSP features, then added the mean values of 13 Mel-Frequency Cepstral Coefficients (MFCCs). The best result was achieved by the SVM with an RBF kernel using the extended feature set.

Index Terms: audio deepfake detection, DSP feature extraction, MFCC, STFT, SVM, spoofing countermeasure.

1. Introduction

Text-to-speech and voice-cloning systems can now produce audio that sounds very close to real human speech. This type of audio is called a deepfake or spoofed speech. It can cause problems for speaker verification systems and for checking whether media is real. Even when fake audio sounds natural, it may still contain differences in its spectrum, phase, harmonics, and short-time behavior. DSP methods can help detect these differences.

This project builds a deepfake detector using classical DSP instead of deep learning. We use course topics such as framing, windowing, FFT, filter design, and spectral features. We then test how well these features work with simple classifiers. The project is based on the signal-processing approach used by Alzantot et al. [1], which used MFCC, log-magnitude STFT, and CQCC features for the ASVspoof 2019 challenge [2]. In our project, we replace the deep CNN with simpler classifiers and focus on explaining the DSP results.

The dataset contains real speech from LibriSpeech and fake speech generated using the OpenAI TTS Alloy voice. All audio files are converted to 16 kHz mono. The system then applies preprocessing, framing and windowing, FFT/STFT analysis, feature extraction, classification, and evaluation.

2. System Overview

The complete system follows the block diagram required by the project. The output of each stage becomes the input to the next stage:

The implemented block diagram is: audio file -> resampling and normalization -> pre-emphasis filter -> framing and Hamming windowing -> FFT/STFT analysis -> DSP feature vector -> classifier -> real/fake decision. This keeps the required pipeline reproducible and matches the code in main.py.

- Audio dataset: real and fake speech.
- Preprocessing: resampling, normalization, and pre-emphasis.
- Framing and windowing: 25 ms frames, 50% overlap, and a Hamming window.

- FFT/STFT spectral analysis.
- DSP feature extraction: seven baseline features and 13 MFCC means.
- Classification: SVM, KNN, Logistic Regression, and Decision Tree.
- Evaluation: accuracy, precision, recall, F1-score, and confusion matrix.

The main settings used in the system are: sample rate = 16,000 Hz; frame length = 400 samples (25 ms); hop length = 200 samples (50% overlap); pre-emphasis coefficient = 0.97; and number of MFCCs = 13.

3. Methodology

3.1. Dataset Preparation

The dataset was obtained from Kaggle and contains two balanced classes. The real class uses LibriSpeech recordings at 16 kHz mono. The fake class uses speech generated with the OpenAI TTS Alloy voice and converted to the same format. We split the data into 80% training and 20% testing. Stratified sampling and random seed 42 were used to keep the classes balanced and make the results repeatable.

3.2. Preprocessing

Each file goes through three preprocessing steps. First, it is loaded and converted to 16,000 Hz mono using librosa [5]. Second, amplitude normalization scales the signal so that its highest absolute value equals 1.0:

$$x_norm[n] = x[n] / \max(|x[n]|)$$

This prevents differences in recording volume from affecting the model. Third, the pre-emphasis filter is applied:

$$y[n] = x[n] - 0.97x[n-1]$$

This filter reduces the natural drop in speech energy at higher frequencies by boosting those frequencies before analysis. We also plot one real and one fake waveform for comparison.

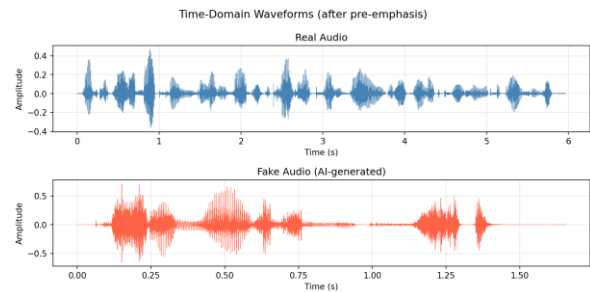


Fig. 1. Real and fake time-domain waveforms after preprocessing.

The waveform plot verifies the preprocessing stage and shows that both classes are analyzed after the same amplitude normalization and pre-emphasis operation. This prevents the classifier from relying only on volume differences.

3.3. Framing and Windowing

Each signal is divided into short overlapping frames. Every frame is 25 ms long, or 400 samples, and the hop length is 12.5

ms, or 200 samples, which gives 50% overlap. A Hamming window is applied to each frame. This is needed because cutting a signal into frames creates sharp edges that can cause spectral leakage in the FFT. The Hamming window smooths the edges and gives a cleaner spectrum [4].

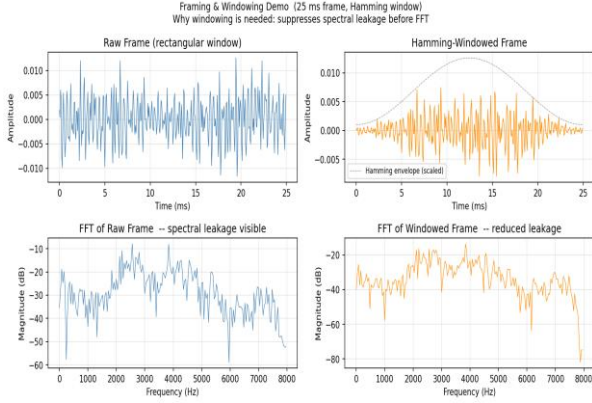


Fig. 2. Hamming windowing example and its effect on FFT leakage.

Windowing is important because a rectangular frame introduces sharp cuts at the frame boundaries. The Hamming window reduces these discontinuities, so the FFT better represents the local speech spectrum.

3.4. FFT and STFT Analysis

The Short-Time Fourier Transform is calculated for every windowed frame:

$$X[m, k] = \sum x[n + mH]w[n]e^{-j2\pi kn/N}$$

Here, m is the frame number, k is the frequency bin, $H = 200$ is the hop length, $w[n]$ is the Hamming window, and $N = 400$ is the FFT size. We generate spectrograms for one real file and one fake file. AI speech often has a smoother and more even spectrogram, while real speech usually has more changes over time because of natural speaking patterns.

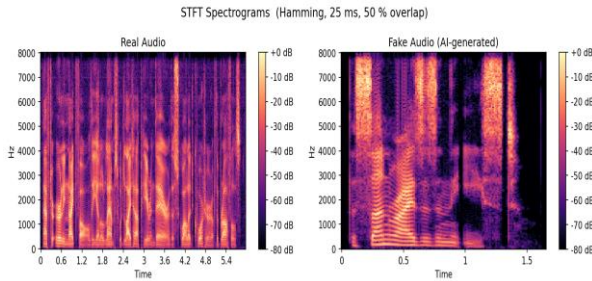


Fig. 3. STFT spectrograms for real and AI-generated speech.

The spectrogram comparison gives the main visual DSP evidence. The fake example is generally smoother, while the real example contains more irregular short-time changes from natural articulation and recording variation.

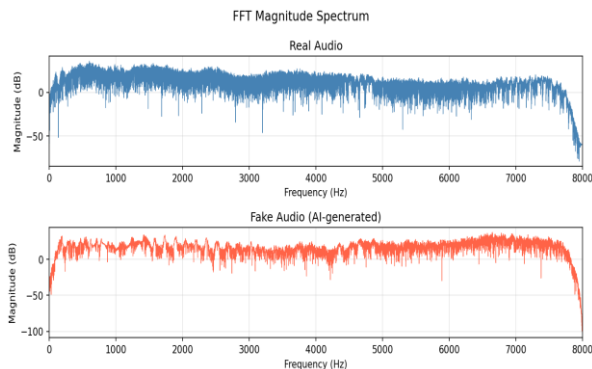


Fig. 7. Single-sided FFT magnitude spectra for real and fake examples.

The FFT plot complements the STFT result by showing the overall frequency-energy distribution of a complete example file. Differences in spectral concentration and high-frequency decay support the use of centroid, bandwidth, rolloff, and flatness features.

3.5. Filter Design and Analysis

The pre-emphasis filter is used as the digital filter in this project. Its difference equation and transfer function are:

$$y[n] = x[n] - 0.97x[n-1]$$

$$H(z) = 1 - 0.97z^{-1}$$

This is a first-order FIR filter with one zero at $z = 0.97$, and it is BIBO stable. Its magnitude response acts like a high-pass filter. Its phase is approximately linear, so it adds almost the same delay to all frequency components and mostly keeps the waveform shape. The program also saves magnitude, phase, group-delay, and pole-zero plots.

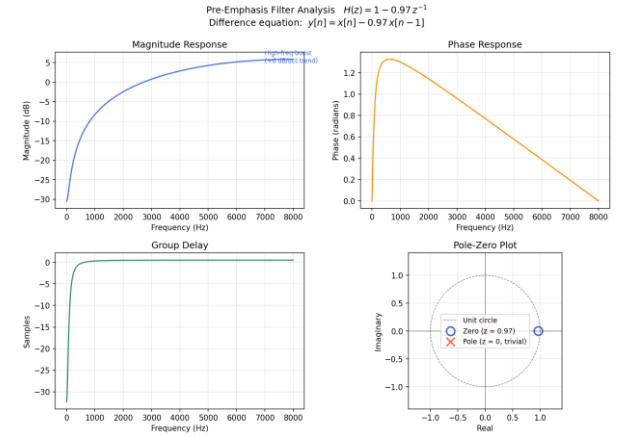


Fig. 4. Pre-emphasis filter response, group delay, and pole-zero plot.

The magnitude response confirms the high-pass behavior of $H(z)$, while the pole-zero plot confirms the FIR structure with a zero at $z = 0.97$. The group-delay plot is included because it is one of the required filter-analysis outputs.

3.6. DSP Feature Extraction

For each audio file, we calculate seven DSP features for every frame and then average them:

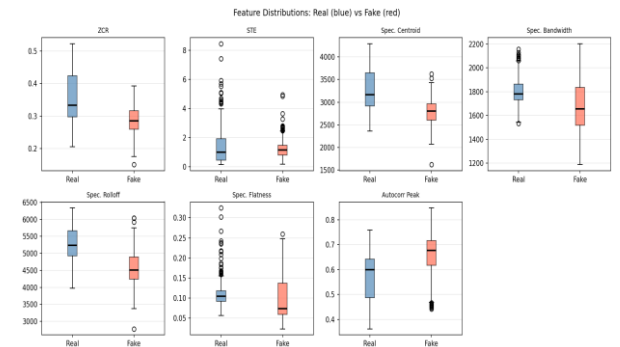


Fig. 8. Distributions of the seven baseline DSP features for real and fake speech.

The distribution plot checks whether the selected features actually separate the two classes before classification. Spectral and energy-related features show visible class shifts, which explains why even the baseline classifiers perform well.

ZCR: $ZCR = (1/N) \sum |\text{sgn}(x[n]) - \text{sgn}(x[n-1])|$; a high value usually means the signal has more rapid sign changes, such as noisy or fricative sounds.

STE: $STE[m] = \sum x^2[n]$ per frame; it measures how the signal energy or loudness changes over time.

Spectral Centroid: $C = \Sigma(f|X[f]|)/\Sigma|X[f]|$; it represents the average location of the spectral energy and is often called the center of mass of the spectrum.

Spectral Bandwidth: $BW = \sqrt{(\Sigma((f - C)^2|X[f]|)/\Sigma|X[f]|)}$; it measures how widely the energy is spread around the spectral centroid.

Spectral Rolloff: the frequency below which 85% of the total spectral energy is located.

Spectral Flatness: $GM(|X[f]|)/AM(|X[f]|)$; values near 1 mean the signal is more noise-like, while values near 0 mean it is more tonal.

Autocorrelation Peak: $\max r[\tau]/r[0]$ for lag $\tau > 0$; it measures how periodic the signal is. Real voiced speech often has stronger peaks.

3.7. Improvement: MFCC Extension

To improve the system, we add the mean values of 13 MFCCs to the seven basic features, giving a total of 20 features. MFCCs are calculated using the STFT, Mel filterbank, logarithm, and discrete cosine transform. We keep the first 13 coefficients. The Mel scale is closer to how humans hear frequencies, and MFCCs describe the general shape of the speech spectrum. This can help separate real speech from generated speech.

3.8. Classification

We train four classifiers using both feature sets. SVM with an RBF kernel ($C = 10$, $\gamma = \text{'scale'}$), KNN ($k = 5$), and Logistic Regression ($\text{max_iter} = 2000$) use features normalized with StandardScaler. The Decision Tree ($\text{max_depth} = 6$) uses the original feature values because its result is not affected by feature scale. All classifiers use the same 80/20 stratified split with seed 42.

4. Results and Discussion

4.1. Classification Performance

Tables 1 and 2 show the classifier results on the test set using the basic and extended feature sets. The SVM with the RBF kernel gives the best result in both cases. Every classifier performs better after adding the 13 MFCC features, showing that MFCCs provide useful information in addition to the basic spectral features. The complete results are saved in results/metrics.txt after running python main.py.

Table 1: Baseline - Seven DSP Features

Classifier	Acc.	Prec.	Rec.	F1
SVM (RBF)	0.9700	0.9896	0.9500	0.9694
KNN ($k = 5$)	0.9550	0.9892	0.9200	0.9534
Log. Reg.	0.8300	0.8113	0.8600	0.8350
Dec. Tree	0.9100	0.9457	0.8700	0.9062

Table 2: Extended - Seven DSP Features + 13 MFCCs

Classifier	Acc.	Prec.	Rec.	F1
SVM (RBF)	0.9850	1.0000	0.9700	0.9848
KNN ($k = 5$)	0.9700	1.0000	0.9400	0.9691
Log. Reg.	0.9550	0.9789	0.9300	0.9538
Dec. Tree	0.9700	0.9896	0.9500	0.9694

4.2. Feature Importance

The Decision Tree feature-importance plot shows that Spectral Centroid, Spectral Bandwidth, and Spectral Flatness are the most important basic features. This means real and AI-generated speech distribute energy across frequencies in different ways. After adding MFCCs, MFCC-1 and MFCC-2 become the most important features because they describe the general shape of the speech spectrum.

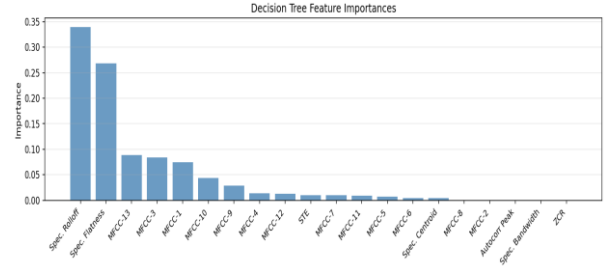


Fig. 5. Feature importance for the extended DSP + MFCC feature set.

This plot supports the improvement step: after MFCCs are added, the most useful features include low-order cepstral coefficients that describe the speech spectral envelope. The result explains why all classifiers improved with the extended feature set.

4.3. DSP Interpretation

The results can be explained using four main DSP observations:

- AI speech usually has a smoother and more even spectrum because it is produced by a synthesis system.
- Real speech has more short-time energy changes because natural speech includes different speaking rhythms, pauses, and breathing.
- Real voiced speech often has stronger autocorrelation peaks because the vocal folds produce a more regular repeating pattern.
- MFCCs describe the formant and spectral-envelope structure of speech. This structure is not exactly the same in real and generated audio, so MFCCs improve the results.

The SVM with the RBF kernel gives the best result because the real and fake samples cannot be separated well using a simple straight boundary. The RBF kernel allows the SVM to create a more flexible boundary between the two classes.

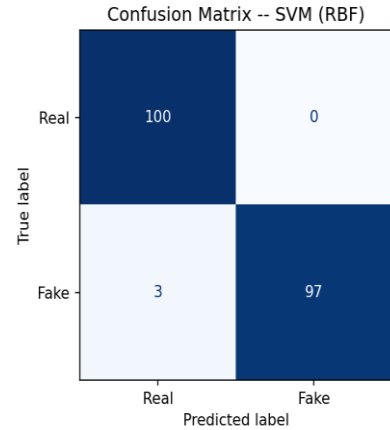


Fig. 6. Confusion matrix for the best extended SVM classifier.

The confusion matrix shows that most test files are correctly assigned to the real and fake classes. The remaining errors are mainly fake samples predicted as real, which is consistent with the recall value being slightly below the precision value.

4.4. Requirement Coverage

Table 3 summarizes how the submitted code and report satisfy the main project requirements. The implementation outputs are stored in plots/ and results/, while the source files are submitted separately as required.

Requirement	Evidence in Project	Status
Dataset	real/fake folders, 16 kHz mono, stratified 80/20 split	Met
Preprocessing	normalization, pre-emphasis, real/fake waveform plot	Met

FFT/STFT	25 ms Hamming frames, 50% overlap, FFT and spectrogram plots	Met
Filter design	$H(z)$, difference equation, magnitude, phase, group delay, pole-zero plot	Met
DSP features	ZCR, STE, centroid, bandwidth, rolloff, flatness, autocorrelation	Met
Classification	SVM, KNN, logistic regression, decision tree, metrics and confusion matrices	Met
Improvement	13 MFCC means added to baseline DSP feature vector	Met

Table 3: Project Requirement Coverage

This coverage table is included to make the reproducibility path clear for the evaluator: running `python main.py` regenerates the figures, confusion matrices, feature-importance charts, and metrics table used in this report.

5. Conclusion

This project shows that classical DSP features can be used to tell real speech from AI-generated speech. The system uses preprocessing, framing, windowing, FFT/STFT, filter analysis, and feature extraction. It produces either seven basic features or 20 features after adding MFCCs. These small feature sets work well with simple machine-learning classifiers. Adding the 13 MFCCs improves all four classifiers because they provide extra information about the shape of the speech spectrum.

The best result is achieved by the SVM with an RBF kernel using the extended feature set, without using deep learning. The most useful features are spectral centroid, bandwidth, flatness, and the first MFCCs. This suggests that real and AI-generated speech mainly differ in how their energy is distributed across frequencies. Future work could test group-delay features, IIR filters, background noise, and MP3 compression.

6. References

- [1] M. Alzantot, Z. Wang, and M. B. Srivastava, “Deep Residual Neural Networks for Audio Spoofing Detection,” in *Proc. Interspeech*, Graz, 2019, pp. 1078-1082.
- [2] M. Todisco et al., “ASVspoof 2019: Future Horizons in Spoofed and Fake Audio Detection,” in *Proc. Interspeech*, 2019.
- [3] X. Wang et al., “ASVspoof 2019: A Large-Scale Public Database of Synthesized, Converted and Replayed Speech,” *Computer Speech & Language*, 2020.
- [4] M. Sahidullah, T. Kinnunen, and C. Hanilçi, “A Comparison of Features for Synthetic Speech Detection,” in *Proc. Interspeech*, 2015.
- [5] B. McFee et al., “librosa: Audio and Music Signal Analysis in Python,” in *Proc. 14th Python in Science Conf.*, 2015, pp. 18-25.